

Lex Programs

To implement and execute multiple LEX programs for tasks such as:

- Validating identifiers
- Counting vowels and consonants
- Replacing specific strings
- Counting lines, words, letters, digits, and special characters

Program 1: Identifier Validation

Algorithm

1. Define regular expressions:
 - `letter` – matches a-z, A-Z, or `_`.
 - `id` – valid identifier: starts with a letter, followed by letters or digits.
2. Match valid identifiers and print confirmation.
3. If it starts with a digit or contains invalid characters, print error message.
4. Stop scanning on newline.

Program

```
%{
#include<stdio.h>
}%

letter [a-zA-Z_]
digit  [0-9]
id      {letter}({letter}|{digit})*

%%
{id}      { printf("Valid identifier: %s\n", yytext); }
[0-9]+    { printf("Invalid identifier (starts with digit): %s\n", yytext); }
.         { printf("Invalid identifier: %s\n", yytext); }
\n        { return 0; }

%%
int main() {
    printf("Enter an identifier: ");
```

```

        yylex();
        return 0;
}

int yywrap() {
    return 1;
}

```

Sample Output

```

Enter an identifier: abc_123
Valid identifier: abc_123

```

Program 2: Count Vowels and Consonants

Algorithm

1. Initialize counters for vowels and consonants.
2. Match vowels and increment the vowel counter.
3. Match alphabetic characters not in vowels and increment consonant counter.
4. Ignore all other characters.

Program

```

%{
#include <stdio.h>
int vowels = 0;
int consonants = 0;
}%

%%
[aeiouAEIOU]    { vowels++; }
[a-zA-Z]        { if (!strchr("aeiouAEIOU", yytext[0])) consonants++; }
.|\\n           { }

%%

int main() {
    printf("Enter a string: ");
    yylex();
    printf("\nTotal number of vowels: %d\n", vowels);
    printf("Total number of consonants: %d\n", consonants);
}

```

```
    return 0;
}
```

Sample Output

```
Enter a string: Lexical
Total number of vowels: 3
Total number of consonants: 4
```

Program 3: Convert "abc" to "ABC"

Algorithm

1. Match the exact string "abc" and print "ABC".
2. Print all other characters as they are.

Program

```
%{
#include <stdio.h>
%}

%%
abc      { printf("ABC"); }
.|\\n    { printf("%s", yytext); }

%%

int main() {
    printf("Enter a string: ");
    yylex();
    return 0;
}
```

Sample Output

```
Enter a string: abcdef
ABCdef
```

Program 4: Count Lines, Words, Letters, Digits, Special Characters

Algorithm

1. Initialize counters for lines, words, letters, digits, and special characters.
2. On newline, increment lines and words.
3. On space or tab, increment word count.
4. On letters, increment letter count.
5. On digits, increment digit count.
6. All other characters are treated as special characters.

Program

```
%{
#include <stdio.h>
int lines = 0, words = 0, letters = 0, digits = 0, sp_ch = 0;
%}

%%
\n          { lines++; words++; }
[ \t]+      { words++; }
[a-zA-Z]    { letters++; }
[0-9]       { digits++; }
.           { sp_ch++; }

%%

int main() {
    printf("Enter input (Ctrl+D to end):\n");
    yylex();
    printf("Lines = %d\n", lines);
    printf("Words = %d\n", words);
    printf("Letters = %d\n", letters);
    printf("Digits = %d\n", digits);
    printf("Special Characters = %d\n", sp_ch);
    return 0;
}

int yywrap() {
    return 1;
}
```

Sample Output

Enter input (Ctrl+D to end):

Hello123!

Lines = 1

Words = 1

Letters = 5

Digits = 3

Special Characters = 1